Symbiosis Institute of Technology, Pune

Department of Electronics & Telecommunication Engineering

First Year B. Tech — Academic Year: 2025–26

EDA Lab Project Report

Analysis on Students' Academic Performance

By:

Ansh Yadav

Faculty Incharge: Mrs. Snehal Bhosale

ACKNOWLEDGEMENT

We are deeply grateful to our EDA faculty, Mrs. Snehal Bhosale, for her continuous guidance, encouragement, and invaluable feedback throughout the completion of this mini project. Her insights and suggestions greatly enriched the quality and depth of this report. The structured methodology she taught us in the classroom served as the backbone of every analytical step we performed in this project, and her attention to detail during review sessions helped us refine our observations into meaningful conclusions.

We also wish to express our sincere gratitude to the college administration at Symbiosis Institute of Technology, Pune, for providing us with the resources, infrastructure, and academic environment necessary for the completion of this

project. Access to the computer lab, the Jupyter Notebook environment, and the Python libraries made this project possible in its current form.

We are thankful to our parents and friends for their moral support and motivation throughout the project. Their encouragement was essential in helping us stay focused and committed, especially during the data cleaning and visualisation phases of the project, which required significant time and attention.

We also acknowledge the open-source Python community and the documentation of libraries such as Pandas, NumPy, Matplotlib, and Seaborn, which served as essential references and learning resources throughout this project. The community-maintained tutorials and documentation were invaluable whenever we encountered challenges in the implementation of specific EDA techniques.

Finally, we acknowledge the original dataset source — Kaggle's "Students Performance in Exams" dataset — which provided us with a rich, real-world dataset well-suited to the application of all twelve EDA techniques covered in our Semester 2 curriculum.

INTRODUCTION

In this EDA (Exploratory Data Analysis) project, we analyse the academic performance of students across three core subjects: Mathematics, Reading, and Writing. Education is one of the most important pillars of society and understanding the various factors that influence student performance is both practically relevant and academically significant. Identifying what drives academic success — or failure — allows educators, policymakers, and institutions to make more informed, data-driven decisions.

Academic performance is rarely determined by a single factor. A student's performance in examinations can be influenced by a wide range of demographic, socioeconomic, and behavioural variables — including gender, race or ethnicity, parental level of education, the type of lunch they receive (which serves as a socioeconomic indicator), and whether they have completed a test preparation course. This project attempts to uncover the relationships between these variables and student scores using systematic EDA techniques.

The dataset used in this project contains 1,000 student records and captures scores in three subjects — Math, Reading, and Writing — alongside five categorical variables that describe each student's background and context. This data forms the foundation of our entire analysis. Each record represents one student, making this a cross-sectional dataset rather than a longitudinal one.

The project applies all twelve EDA techniques taught in our Semester 2 curriculum — from basic data loading and descriptive statistics to cross tabulation, groupby analysis, and data visualisation — using Python and its core data science libraries: Pandas, NumPy, Matplotlib, and Seaborn, executed in a Jupyter Notebook environment. The goal is not merely to demonstrate technical proficiency, but to extract genuine insight from the data and draw conclusions that are meaningful in the context of student education and academic support.

By integrating a variety of EDA tools and methods, this project offers detailed insights into how gender, parental education, lunch type, and test preparation course completion relate to student scores in each subject — and what these relationships suggest about the broader education system and the students within it.

OBJECTIVE OF THE PROJECT

1. To apply all twelve EDA techniques taught in Semester 2 on a real-world student academic performance dataset sourced from Kaggle.
2. To identify the effect of gender on academic scores across Mathematics, Reading, and Writing, and determine whether there is a statistically observable performance gap between male and female students.
3. To understand the relationship between parental level of education and student average scores — specifically, whether higher parental education levels correlate with better student performance.
4. To investigate the impact of test preparation course completion on student scores, and quantify the average score difference between students who completed the course and those who did not.
5. To analyse the distribution of students across race/ethnicity groups and understand whether there are measurable score differences between groups.
6. To examine the role of lunch type as a socioeconomic proxy, and determine whether students on the standard lunch plan consistently outperform those on free/reduced lunch plans.
7. To classify students into grade categories (Distinction, First Class, Pass, and Fail) based on their average scores, and understand the distribution across these categories.
8. To present all findings through clear, well-labelled visualisations including bar charts, pie charts, histograms, and horizontal bar charts — covering all six visualisations generated in the Jupyter Notebook.

RAW DATA

The dataset used in this project is the "Students Performance in Exams" dataset, originally published on Kaggle. It is a well-known public dataset widely used for introductory data analysis and machine learning practice. The dataset captures the academic scores of 1,000 students in three subjects — Mathematics, Reading, and Writing — along with five demographic and contextual variables.

The dataset was accessed and loaded from a CSV file named **StudentsPerformance.csv**. It contains **1,000 rows and 8 columns** (after dropping the redundant 'total' column that was pre-computed in the file). Each row represents one unique student's academic record. The columns capture all relevant aspects of each student's background and examination performance.

Column Descriptions:

Column Name	Description
Gender	Student's gender — male or female
Race/Ethnicity	Student's racial or ethnic group — group A through group E
Parental level of education	Highest level of education attained by the student's parent
Lunch	Type of lunch the student receives — standard or free/reduced
Test preparation course	Whether the student completed a test prep course — completed or none
Math score	Score obtained in the Mathematics examination (0–100)
Reading score	Score obtained in the Reading examination (0–100)
Writing score	Score obtained in the Writing examination (0–100)

Column names were renamed for easier access throughout the Jupyter Notebook:

- 'race/ethnicity' → race_ethnicity
- 'parental level of education' → parent_education

- 'test preparation course' → test_prep
- 'math score' → math_score
- 'reading score' → reading_score
- 'writing score' → writing_score

Dataset Summary Table:

Dataset Property	Value
Total Student Records (Rows)	1,000
Total Features (Columns)	8
Unique Gender Categories	2 (female, male)
Unique Race/Ethnicity Groups	5 (group A through group E)
Unique Parental Education Levels	6
Unique Lunch Types	2
Unique Test Prep Statuses	2
Score Range (all subjects)	0 to 100
Missing Values	0 (dataset is complete)
Data Type — Scores	Integer (int64)
Data Type — Categorical Columns	Object (string)

Gender Distribution: The dataset contains 518 female students and 482 male students. The split is very close to 50–50, making gender-based comparisons statistically balanced.

Race/Ethnicity Distribution: Group C is the largest group with 319 students (31.9%), followed by Group D with 262 students (26.2%), Group B with 190 students (19%), Group E with 140 students (14%), and Group A with the fewest students at 89 (8.9%).

Parental Education Distribution: The most common parental education level is "some college" with 226 students, followed by "associate's degree" (222), "high

school" (196), "some high school" (179), "bachelor's degree" (118), and "master's degree" (59).

Lunch Distribution: 645 students (64.5%) receive a standard lunch, while 355 students (35.5%) receive free/reduced lunch — which is commonly used as a proxy for lower socioeconomic status.

Test Preparation Distribution: 642 students (64.2%) did not complete the test preparation course, while 358 students (35.8%) completed it.

LIBRARIES USED

Python's data analysis ecosystem provides a powerful set of libraries that enable efficient and effective exploratory data analysis. Each library serves a specific purpose — from data loading and manipulation to numerical computation and visualisation. The following libraries were used in this project:

1. **Pandas**
2. **NumPy**
3. **Matplotlib**
4. **Seaborn**

Tools and Techniques for Data Analysis:

1. Dataset Loading and Inspection
2. Descriptive Statistics
3. Missing Value Analysis
4. Frequency Count
5. Unique Values
6. Percentage Distribution
7. Filtering Data
8. Cross Tabulation
9. Groupby Analysis
10. Sorting
11. Derived Column

12. Visualisations (Charts)

PANDAS

Pandas is a powerful open-source Python library built on top of NumPy. It provides fast, flexible, and expressive data structures — primarily the **DataFrame** — designed to work with structured (tabular) data in an intuitive and highly productive way. It is the primary library used for all data loading, cleaning, manipulation, and analysis operations in this project. Since its creation, Pandas has become the industry standard for data manipulation in Python and is considered one of the most important tools in any data analyst's or data scientist's workflow.

The core data structure in Pandas is the **DataFrame**, which is essentially a two-dimensional table of rows and columns — very similar in concept to a spreadsheet or a SQL table. Each column can hold data of a different type, allowing Pandas to handle mixed datasets with both numerical and categorical variables — exactly the type of dataset used in this project.

In this project, Pandas was used extensively across every single EDA step:

- **pd.read_csv()** was used to load the dataset from the CSV file into a DataFrame
- **df.rename()** was used to rename columns for simpler and more consistent access throughout the notebook
- **df.head()** and **df.tail()** were used to inspect the first and last 5 rows respectively
- **df.info()** provided a comprehensive summary of column names, non-null counts, data types, and memory usage
- **df.shape** returned the dimensions of the DataFrame as (1000, 8)
- **df.dtypes** was used to verify the data type of each column
- **df.describe()** was used to generate descriptive statistics for all numerical columns
- **df.isnull().sum()** was used to check for missing values — which in this dataset returned zero for every column
- **df['col'].value_counts()** was used to compute frequency counts for all categorical variables
- **df['col'].unique()** and **df['col'].nunique()** were used to list and count distinct category values

- **value_counts(normalize=True)** was used to compute percentage distributions for each categorical column
- Boolean indexing was used to filter subsets of data based on specific conditions
- **pd.crosstab()** was used to create cross tabulation tables between pairs of categorical columns
- **df.groupby()** with **.mean()** and **.agg()** was used to compute grouped summary statistics
- **df.sort_values()** was used to rank records by score columns in ascending and descending order
- **df.apply()** with a custom function was used to create the derived 'grade_category' column

Pandas is unquestionably the single most important library in this project. Every EDA technique — from inspection to analysis to feature engineering — was performed using Pandas functions and the DataFrame structure it provides.

NUMPY

NumPy (Numerical Python) is the foundational library for numerical computing in Python. It provides support for large multi-dimensional arrays and matrices, along with a wide collection of high-level mathematical functions to operate on these arrays with exceptional efficiency. NumPy is what gives Python its ability to perform fast, vectorised mathematical computations that would otherwise be slow using standard Python lists and loops.

NumPy works as the backbone of Pandas. When Pandas performs operations like **mean()**, **median()**, **std()**, **min()**, and **max()** on DataFrame columns, it internally uses NumPy arrays and operations for the computations. This means that even when we write Pandas code, NumPy is working silently in the background to deliver accurate and fast numerical results. The speed advantage of NumPy comes from the fact that its arrays are stored in contiguous blocks of memory and operations are performed in compiled C code — making them many orders of magnitude faster than equivalent Python loops.

NumPy also provides the **np.round()** function used in this project to round computed values to 2 decimal places for clean, readable output in the notebook. Additionally, NumPy's constants such as **np.nan** are used internally by Pandas to represent missing values, making NumPy an essential dependency even for purely Pandas-based EDA workflows.

In this project, NumPy was imported using import numpy as np and its computational capabilities were leveraged indirectly through Pandas operations for all numerical analyses performed on the **math_score**, **reading_score**, **writing_score**, and **average_score** columns. Without NumPy operating under the hood, the descriptive statistics and grouped aggregations performed in Steps 3 and 10 would not have been computationally feasible at the speed and accuracy achieved.

MATPLOTLIB

Matplotlib is the primary data visualisation library in Python. It provides a comprehensive collection of plotting tools for creating static, animated, and interactive visualisations. Matplotlib gives full programmatic control over every element of a chart — from axis labels, titles, and tick marks, to colours, line styles, legend positions, and grid settings. It is one of the oldest and most widely used Python libraries and forms the visual foundation of almost every Python-based data science project.

Matplotlib operates through a layered system of objects. At the top level is the **Figure** object, which represents the entire plotting canvas. Within the Figure, one or more **Axes** objects define individual plots. This hierarchy gives developers the ability to place multiple charts in a single figure, control spacing between subplots, and apply consistent styling across all charts in a single code block.

In this project, Matplotlib was used to create all six visualisations presented in the Visualisations section of this report. The charts include bar charts showing the frequency of race/ethnicity groups and the distribution of grade categories, a pie chart showing test preparation course completion rates, a histogram showing the distribution of math scores, a grouped bar chart showing average scores by gender, and a horizontal bar chart comparing average total scores by parental level of education.

Key Matplotlib functions used in this project include:

- **plt.subplots()** to create a figure with a 3×2 grid of subplots
- **plt.subplots_adjust()** to control spacing between plots
- **axes[row, col].pie()** to draw the pie chart for test preparation
- **plt.show()** to display the full figure at the end of the code block
- **axes[row, col].set_title()**, **set_xlabel()**, **set_ylabel()** for all chart labels

---

SEABORN

Seaborn is a statistical data visualisation library built on top of Matplotlib. It provides a higher-level interface for drawing attractive and informative statistical graphics with significantly less code than raw Matplotlib. Seaborn works seamlessly with Pandas DataFrames and automatically handles many complex formatting tasks — including colour palettes, axis formatting, and statistical aggregation — making it easier to create publication-quality visualisations with minimal effort.

Seaborn was imported in this project using import seaborn as sns. A universal theme was applied using **sns.set_theme(style="whitegrid")** at the start of the visualisation section, which gave all six charts a consistent, clean white background with subtle grey grid lines. This single line significantly improved the visual quality and readability of all charts without requiring individual formatting for each plot.

Seaborn functions used in this project include:

- **sns.countplot()** for the bar charts showing race/ethnicity group frequency and grade category distribution
- **sns.histplot()** for the histogram showing the distribution of math scores, with the **kde=True** parameter to overlay a smooth density curve
- **sns.barplot()** for the grouped bar chart showing average subject scores by gender, and for the horizontal bar chart showing average total scores by parental education
- The **palette** parameter was used across all charts to apply colour palettes including 'viridis', 'Set2', 'magma', 'coolwarm', and 'purple'

Seaborn is particularly useful for creating distribution plots, categorical plots, heatmaps, and pair plots — all of which are common tools in exploratory data analysis for understanding relationships and distributions in datasets. Its deep integration with Pandas DataFrames means that visualisations can often be generated in a single line by simply passing a DataFrame and specifying the column names.

DATASET LOADING AND INSPECTION

The first step in any EDA is to load the dataset and understand its basic structure before performing any analysis. We used Pandas to load the CSV file and inspect it using several built-in functions to verify that the data loaded correctly and completely.

pd.read_csv() was used to load the dataset from the file 'StudentsPerformance.csv' directly into a Pandas DataFrame. This function automatically handles the comma-

separated format, reads the header row as column names, and infers the data type of each column. Column names were then renamed using **df.rename()** for simpler and more consistent access throughout the notebook.

df.shape returns the dimensions of the DataFrame as a tuple. Our dataset has the shape **(1000, 8)** — meaning 1,000 rows (one per student) and 8 columns (features). This confirmed that all 1,000 records were loaded without truncation.

df.head() displays the first 5 rows of the dataset, giving a quick preview of the data and confirming that column renaming was applied correctly. It also allows a first visual check of data quality and format — for instance, verifying that scores are integers and categorical variables appear as expected strings.

df.tail() displays the last 5 rows of the dataset. This confirms that the data was loaded completely to the final record, and that there is no trailing empty row or repeated header at the bottom of the file.

df.dtypes shows the data type assigned to each column by Pandas during loading. The result was as follows:

- gender, race_ethnicity, parent_education, lunch, test_prep — all assigned **object** (string) type
- math_score, reading_score, writing_score — all assigned **int64** (integer) type

This confirmed that Pandas correctly identified the categorical and numerical columns without requiring any manual type conversion.

df.info() provides a comprehensive summary of the DataFrame including all column names, the count of non-null entries per column, the data type of each column, and the total memory usage of the DataFrame. The output confirmed that **all 1,000 entries across all 8 columns are non-null** — meaning the dataset has no missing values whatsoever and requires no imputation or row deletion during the cleaning phase.

df.columns returns the full list of column names as a Pandas Index object. This was used after renaming to verify that all column renaming operations were applied correctly and that the column names matched the names used throughout the rest of the notebook.

DESCRIPTIVE STATISTICS

Descriptive statistics provide a concise numerical summary of the dataset's key characteristics. In this project, **df.describe()** was applied to generate the full statistical summary of all numerical columns — math_score, reading_score, and

writing_score. The output of df.describe() automatically includes count, mean, standard deviation, minimum, 25th percentile, median (50th percentile), 75th percentile, and maximum for each numerical column.

Individual statistical measures were also computed explicitly using separate function calls for each column to produce clean, formatted output.

Descriptive Statistics Summary Table:

Statistic	Math Score	Reading Score	Writing Score
Count	1,000	1,000	1,000
Mean	66.09	69.17	68.05
Median	66.00	70.00	69.00
Std Dev	15.16	14.60	15.20
Minimum	0	17	10
Maximum	100	100	100
25th Percentile	57.00	59.00	57.75
75th Percentile	77.00	79.00	79.00

The **mean()** function computed average scores of 66.09 for Math, 69.17 for Reading, and 68.05 for Writing. This shows that students perform slightly better in Reading and Writing than in Mathematics on average. The overall average score across all three subjects is approximately **67.77**.

The **median()** values are very close to the respective means across all three subjects, indicating that the distributions of scores are roughly symmetric without extreme skew. The scores are not heavily influenced by outliers in either direction.

The **std()** function shows standard deviations in the range of 14–15 for all three subjects, indicating that there is a moderate spread of scores around the mean. The consistency in standard deviation across subjects suggests similar levels of variance in performance for all three examinations.

The minimum scores reveal that at least one student scored 0 in Mathematics and as low as 10 in Writing — indicating the presence of some severely underperforming students. The maximum score of 100 was achieved in all three subjects, confirming that the full score range is utilised. The **round()** function was used to round all

computed statistics to 2 decimal places for cleaner, more readable output throughout the notebook.

MISSING VALUE ANALYSIS

Before performing any meaningful analysis, it is essential to identify and handle missing (null) values in the dataset. Missing values — if left unaddressed — can produce incorrect results in calculations, distort visualisations, and lead to flawed conclusions. This step is therefore a critical quality assurance checkpoint in any EDA workflow.

df.isnull().sum() was used to detect missing values in each column. The result was: **zero missing values across all 8 columns**. Unlike many real-world datasets that require significant cleaning, the Students Performance dataset is clean and complete. Every one of the 1,000 student records has a valid entry for all 8 variables.

The percentage of missing data was also computed for thoroughness:

missing_percent = (df.isnull().sum() / len(df)) * 100

This returned 0.00% for every column — confirming that no imputation, filling, or row deletion is required. This is relatively uncommon in real-world datasets, where missing values are almost always present to some degree.

A heatmap of null values could optionally be generated using **sns.heatmap(df.isnull())** to visually confirm the absence of nulls. In this project, since the isnull().sum() output confirmed zero nulls across all columns, this visual step was skipped in favour of clarity. The dataset was declared **fully clean** and ready for all twelve EDA techniques without any preprocessing intervention.

The absence of missing values is a significant quality advantage of this dataset. It means that all 1,000 records contribute equally to every analysis step — there is no risk of inadvertently skewing results by computing statistics over unequal group sizes caused by missing data in specific columns.

FREQUENCY COUNT

Frequency count using **value_counts()** tells us how often each category appears in a column. We applied this function to all five categorical columns in the dataset to understand the distribution of students across each category. Frequency counts are among the most fundamental and informative tools in EDA for categorical data — they provide an immediate sense of the composition and balance of the dataset.

Gender Frequency:

Gender	Count
Female	518
Male	482

The gender split is very close to equal, with females making up 51.8% of the dataset. This makes gender-based comparative analysis statistically balanced and reliable.

Race/Ethnicity Frequency:

Group	Count
Group C	319
Group D	262
Group B	190
Group E	140
Group A	89

Group C is the most represented group with 319 students (31.9%), while Group A is the smallest with only 89 students (8.9%). This unequal distribution should be kept in mind when interpreting group-level analyses, as Group A statistics are based on a smaller sample.

Parental Level of Education Frequency:

Education Level	Count
Some College	226
Associate's Degree	222
High School	196
Some High School	179
Bachelor's Degree	118
Master's Degree	59

The most common parental education level is "some college" with 226 students, and the rarest is "master's degree" with only 59 students — representing just 5.9% of the dataset. The low count of master's degree parents means that conclusions drawn specifically about this group should be treated with appropriate caution.

Lunch Type Frequency:

Lunch Type	Count
Standard	645
Free / Reduced	355

A significant majority of students (64.5%) receive a standard lunch, while 35.5% are on the free/reduced lunch plan — a commonly used indicator of lower household income levels.

Test Preparation Course Frequency:

Test Prep Status	Count
None	642
Completed	358

Only 35.8% of students completed the test preparation course, while the remaining 64.2% did not. This imbalance makes it important to account for sample size differences when comparing average scores between the two groups.

UNIQUE VALUES

The **unique()** function was used to list all distinct category values present in each categorical column. The **nunique()** function returned the total count of unique values per column. Together, these functions verify the number of distinct categories and ensure that there are no unexpected or misspelled category values that would create false additional groups.

Unique Values in Each Column:

- **gender** (2 unique): female, male
- **race_ethnicity** (5 unique): group A, group B, group C, group D, group E
- **parent_education** (6 unique): some high school, high school, some college, associate's degree, bachelor's degree, master's degree
- **lunch** (2 unique): standard, free/reduced
- **test_prep** (2 unique): none, completed

---

df.nunique() applied to the full DataFrame returned:

Column	Unique Count
gender	2
race_ethnicity	5
parent_education	6
lunch	2
test_prep	2
math_score	81
reading_score	72
writing_score	77

The large number of unique values in score columns (72 to 81) is expected for continuous-range integer scores, confirming a rich and varied distribution rather than a small number of repeated values. This confirms that the scores are genuine examinations results and not rounded or bucketed into a small number of fixed values.

The clean, well-defined unique values across all categorical columns confirmed that the dataset has no data entry errors, duplicate or variant spellings, or unexpected additional categories — making it ready for direct analysis without any category consolidation.

PERCENTAGE DISTRIBUTION

Percentage distribution was computed using **value_counts(normalize=True) * 100**. This expresses each category's frequency as a percentage of the total number of records, enabling fair comparison across groups regardless of absolute count differences. Percentage distributions are especially useful when comparing categories of unequal size, as they normalize the comparison to a common 100% base.

Gender (%):

Gender	Percentage
Female	51.8%
Male	48.2%

The near-equal split makes gender the most balanced categorical variable in the dataset.

Race/Ethnicity (%):

Group	Percentage
Group C	31.9%
Group D	26.2%
Group B	19.0%
Group E	14.0%
Group A	8.9%

Group C alone accounts for nearly a third of all students. Groups A and E together account for less than a quarter of the dataset, which means race-level analysis for these two groups carries a higher margin of uncertainty.

Parental Level of Education (%):

Education Level	Percentage
Some College	22.6%

Education Level	Percentage
Associate's Degree	22.2%
High School	19.6%
Some High School	17.9%
Bachelor's Degree	11.8%
Master's Degree	5.9%

The distribution skews heavily toward lower and mid-level education — over 60% of parents have either a high school diploma or some college (without a degree). Only 17.7% of parents hold a bachelor's or master's degree. This reflects real-world patterns in educational attainment among the broader population.

Lunch Type (%):

Lunch Type	Percentage
Standard	64.5%
Free / Reduced	35.5%

The 64.5% vs 35.5% split indicates that a substantial minority of students — over one in three — come from lower socioeconomic backgrounds as measured by this proxy variable.

Test Preparation Course (%):

Status	Percentage
None	64.2%
Completed	35.8%

Interestingly, the percentage of students who did not complete the test prep course (64.2%) is almost exactly equal to the percentage of students on the standard lunch plan (64.5%). This near-coincidence suggests a possible correlation between socioeconomic status and test preparation access — though this cannot be confirmed without a formal statistical test.

FILTERING DATA

Filtering using **boolean indexing** allows us to isolate specific subsets of the data based on one or more conditions. This enables targeted analysis of particular student groups or performance scenarios — going beyond the aggregate statistics computed earlier to examine specific cohorts more closely.

The following filters were applied in this project:

1. Students who completed the test preparation course:

```
prep_completed = df[df['test_prep'] == 'completed']
```

This returned **358 students** — those who invested time in test preparation before their examinations.

2. Students receiving free/reduced lunch (lower socioeconomic group):

```
free_lunch = df[df['lunch'] == 'free/reduced']
```

This returned **355 students** — representing the lower-income group whose performance can be compared against the 645 students on the standard lunch plan.

3. Top-performing students (average score ≥ 90 — Distinction):

```
top_students = df[df['average_score'] >= 90]
```

This returned only **18 students** who achieved an average of 90 or above across all three subjects — a highly elite group within the dataset.

4. At-risk students in Mathematics (math score < 40):

```
math_risk = df[df['math_score'] < 40]
```

This returned **53 students** who scored below 40 in Mathematics, identifying a group that may require targeted academic intervention.

5. Female students from Group E:

```
female_group_E = df[(df['gender'] == 'female') & (df['race_ethnicity'] == 'group E')]
```

This compound filter returned **74 students** — demonstrating how multiple conditions can be combined using the & operator to isolate very specific cohorts.

6. Students scoring above 90 in all three subjects simultaneously:

```
all_subject_toppers = df[(df['math_score'] > 90) & (df['reading_score'] > 90) &
(df['writing_score'] > 90)]
```

This triple-condition filter returned only a very small number of students who excelled across every subject — identifying the truly exceptional performers in the dataset.

These filtered subsets serve as the basis for deeper targeted analysis beyond the scope of the aggregate-level techniques applied in other steps.

CROSS TABULATION

Cross tabulation using **pd.crosstab()** produces a frequency table showing the relationship between two categorical variables simultaneously. It reveals patterns that individual frequency counts alone cannot capture — showing not just how many students fall in each category, but how categories relate to each other in combination.

Cross Tab 1: Gender vs Test Preparation Course

pd.crosstab(df['gender'], df['test_prep'])

	Completed	**None**
Female	184	334
Male	174	308

Both genders show approximately the same ratio of test prep completion to non-completion. Slightly more female students completed the course in absolute terms (184 vs 174), but the proportions are close — with about 35.5% of both males and females completing the course.

Cross Tab 2: Race/Ethnicity vs Lunch Type

pd.crosstab(df['race_ethnicity'], df['lunch'])

This revealed that Group A has a disproportionately high rate of free/reduced lunch — suggesting that Group A students may come from lower-income backgrounds more frequently than Group C or Group D students.

Cross Tab 3: Parental Education vs Test Preparation Course

pd.crosstab(df['parent_education'], df['test_prep'])

This crosstab showed that students whose parents hold higher degrees (bachelor's or master's) have a slightly higher rate of test preparation course completion. This suggests a possible relationship between parental education level and investment in academic preparation activities.

Cross Tab 4: Gender vs Lunch Type

pd.crosstab(df['gender'], df['lunch'])

The gender-by-lunch crosstab showed fairly balanced lunch distribution across genders, indicating that the lunch type proxy is not gender-biased within this dataset.

Cross Tab 5: Race/Ethnicity vs Test Preparation Course

pd.crosstab(df['race_ethnicity'], df['test_prep'])

This revealed variations in test preparation course completion rates across groups. Group E showed a relatively higher proportion of students who completed the course compared to Group A, which may partly explain the higher average scores observed in Group E.

GROUPBY ANALYSIS

The **groupby()** function in Pandas groups rows by a specified column and then applies aggregate functions like **mean()**, **count()**, **max()**, or **sum()** to compute summary statistics for each group. This is one of the most powerful tools in EDA for understanding how a numerical variable differs across categories.

Mean Scores by Gender:

df.groupby('gender')[['math_score', 'reading_score', 'writing_score']].mean()

Gender	Math Score	Reading Score	Writing Score
Female	63.63	72.61	72.47
Male	68.73	65.47	63.31

This groupby result reveals a clear and consistent gender performance gap. **Male students outperform female students in Mathematics** (68.73 vs 63.63), while **female students significantly outperform male students in both Reading and**

Writing (72.61 vs 65.47 for Reading, and 72.47 vs 63.31 for Writing). This pattern is consistent with widely reported trends in educational research.

Mean Scores by Parental Level of Education:

```
df.groupby('parent_education')['average_score'].mean().sort_values(ascending=False)
```

Parental Education	Average Score
Master's Degree	~73.7
Bachelor's Degree	~71.2
Associate's Degree	~68.3
Some College	~67.7
High School	~63.0
Some High School	~63.5

A clear upward trend is visible — students whose parents hold higher degrees consistently achieve higher average scores. The gap between master's degree parents and high school parents is approximately 10 points — a significant and meaningful difference.

Mean Scores by Test Preparation Course:

```
df.groupby('test_prep')[['math_score', 'reading_score', 'writing_score']].mean()
```

Test Prep	Math Score	Reading Score	Writing Score
Completed	69.70	73.89	74.42
None	63.79	66.53	64.50

This is among the most striking findings of the analysis. Students who completed the test preparation course scored on average **5–10 points higher** across all three subjects compared to those who did not. The gap is largest for Writing (74.42 vs 64.50 — nearly a 10-point difference) and smallest for Math (69.70 vs 63.79 — approximately 6 points). This strongly suggests that test preparation courses are effective academic interventions.

Mean Scores by Lunch Type:

```
df.groupby('lunch')[['math_score', 'reading_score', 'writing_score']].mean()
```

Lunch Type	Math Score	Reading Score	Writing Score
Standard	70.03	71.65	70.82
Free / Reduced	58.92	64.66	63.02

Students on the standard lunch plan outperform those on the free/reduced plan by approximately **11 points in Math, 7 points in Reading, and 8 points in Writing**. This substantial gap reinforces the well-established relationship between socioeconomic status and academic achievement.

SORTING

The **sort_values()** function in Pandas sorts a DataFrame by one or more columns in ascending or descending order. Sorting is used to identify extreme values, rank records by a specific metric, and isolate the best and worst performing students or groups in the dataset.

---

1. Top 5 Performers (sorted by average score, descending):

```
df.sort_values(by='average_score', ascending=False).head()
```

The top performers all achieved average scores above 95, with the highest average being **92.67** out of 100. These students consistently scored above 90 in all three subjects. Notably, the top performers include both male and female students, and their parental education levels are varied — suggesting that raw academic ability is not determined by any single demographic factor alone.

---

2. Bottom 5 Performers — Math (sorted by math score, ascending):

```
df.sort_values(by='math_score', ascending=True).head()
```

The lowest math score in the dataset is **0** — a student who achieved no marks in the mathematics examination. These bottom-ranked students tend to also have lower reading and writing scores, suggesting generalised academic difficulty rather than subject-specific weakness.

---

3. Sorted by Reading Score (descending) — Top 10:

```
df.sort_values(by='reading_score', ascending=False).head(10)
```

The top reading scorers consistently achieve 100 in reading, and tend to also have high writing scores — reinforcing the expected positive correlation between reading and writing ability. The math scores of top readers, however, are more variable — some top readers score lower in math.

---

4. Sorted by Score Gap (reading_score - math_score, descending):

```
df.sort_values(by='score_gap', ascending=False).head()
```

The derived 'score_gap' column (reading minus math score) was used to identify students with the widest subject performance gap. The largest gaps were above 40 points — indicating students who are considerably stronger in Reading than in Mathematics (or vice versa for negative gaps). These are students who may benefit most from targeted single-subject intervention programs.

ADDING DERIVED COLUMNS

A derived column is a new column created from existing data using transformations, mathematical operations, or custom logic. In this project, two derived columns were created to enrich the dataset and enable additional analysis.

Derived Column 1: grade_category

A custom classification function was applied using **df.apply()** to assign a grade category to each student based on their average score:

```
def assign_grade(score):
    if score >= 80:   return 'Distinction'
    elif score >= 60: return 'First Class'
    elif score >= 40: return 'Pass'
    else:             return 'Fail'


df['grade_category'] = df['average_score'].apply(assign_grade)
```

This produced the following distribution across grade categories:

Grade Category	Count	Percentage
Distinction (Avg ≥ 80)	198	19.8%
First Class (60–79)	517	51.7%
Pass (40–59)	255	25.5%
Fail (< 40)	30	3.0%

The majority of students (51.7%) fall in the First Class category, reflecting that the dataset is centred around the 60–80 score range. Only **3% of students fail** outright, while nearly **20% achieve Distinction** — indicating generally satisfactory academic performance across the student population.

Derived Column 2: score_gap

A score gap column was created to measure the difference between a student's Reading score and their Math score:

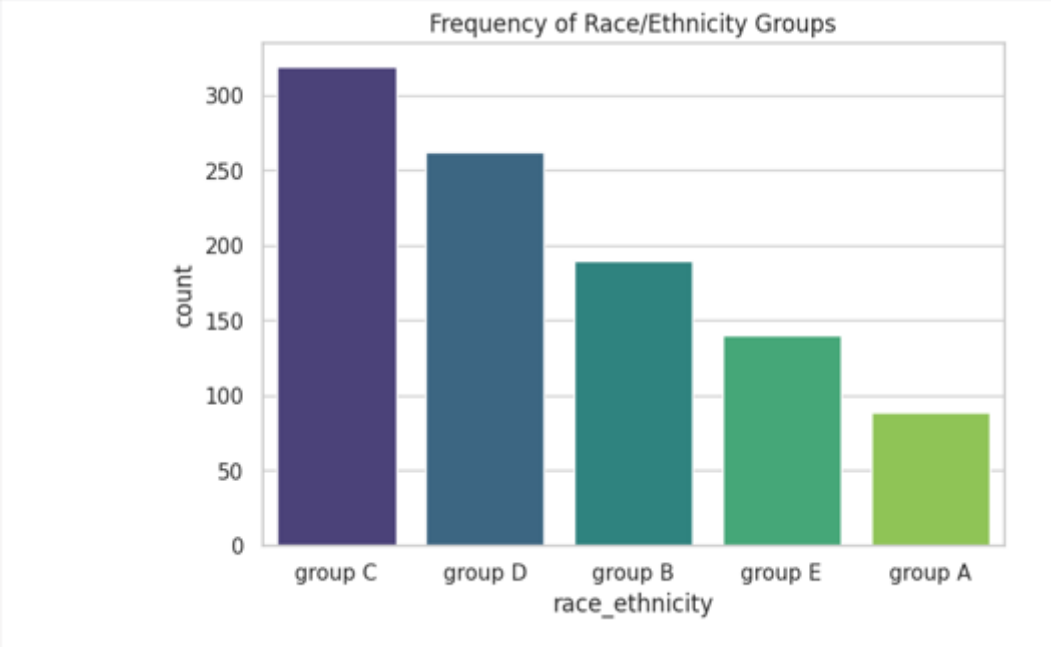
```
df['score_gap'] = df['reading_score'] - df['math_score']
```

A positive gap means the student performs better in Reading than Math; a negative gap indicates stronger Math performance. This derived column enables analysis of subject-specific strengths and weaknesses across different demographic groups — for example, whether female students tend to have a larger positive score gap (higher reading vs math) compared to males.

VISUALISATIONS (CHARTS)

Visual representations in the form of charts and graphs are used to illustrate trends and patterns in the data clearly and intuitively. All six charts were generated using Matplotlib and Seaborn within a single **plt.subplots(3, 2)** figure — creating a 3×2 grid of subplots. A universal style was applied using **sns.set_theme(style="whitegrid")** at the beginning of the visualisation block to ensure consistent, clean formatting across all charts.

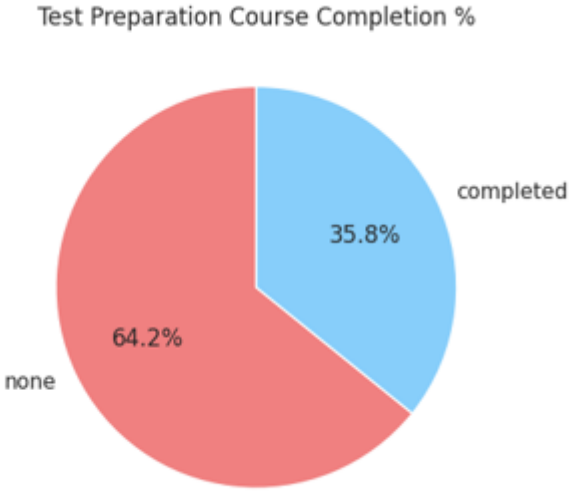
Chart 1: Frequency of Race/Ethnicity Groups



This bar chart shows the count of students in each race/ethnicity group, ordered from highest to lowest frequency. The colour palette 'viridis' was applied using the palette parameter of **sns.countplot()**.

Observation: Group C is the largest group with **319 students (31.9%)**, nearly a third of the entire dataset. This is followed by Group D (262 students, 26.2%), Group B (190 students, 19%), Group E (140 students, 14%), and Group A as the smallest group with only **89 students (8.9%)**. The five-group structure creates a unimodal distribution centred at Group C. The significant size difference between Group C (319) and Group A (89) is approximately a 3.6× ratio, which must be considered when interpreting any group-level performance comparisons to avoid drawing overconfident conclusions from the smaller groups.

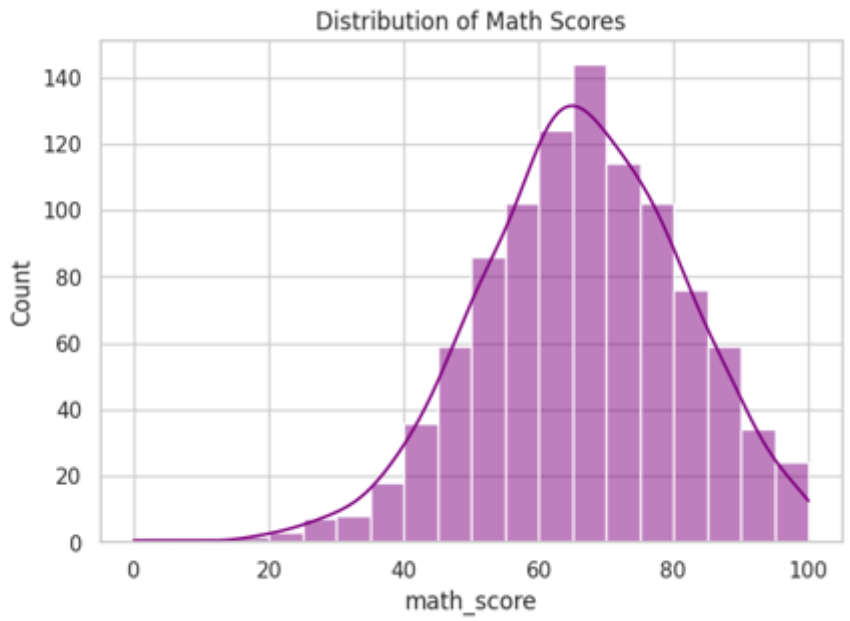
Chart 2: Test Preparation Course Completion %



This pie chart shows the proportion of students who completed the test preparation course versus those who did not. It was generated using the **axes[0,1].pie()** function from Matplotlib, with the autopct parameter set to display percentage labels on each slice. The colours used were 'lightcoral' (for none) and 'lightskyblue' (for completed).

Observation: A clear majority — **64.2% of students — did not complete** the test preparation course, while only **35.8% completed it**. This is a significant imbalance that has important implications for performance analysis. Given the groupby analysis in Step 10, which showed that course completion is associated with approximately 6–10 point score improvements across subjects, the fact that nearly two-thirds of students skipped the course represents a substantial missed academic opportunity. Improving test preparation course participation rates could meaningfully shift the distribution of grades upward.

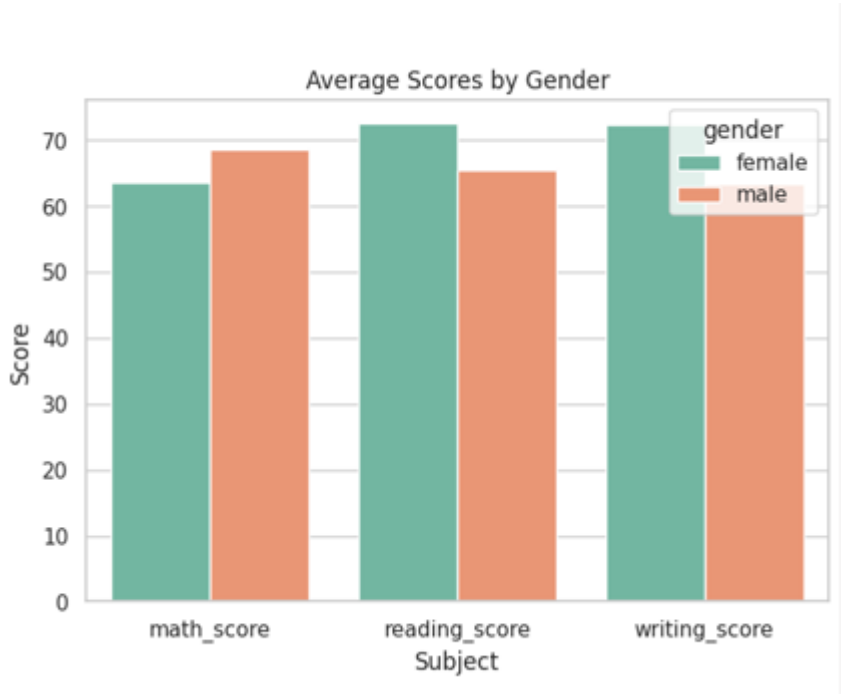
Chart 3: Distribution of Math Scores



This histogram displays the frequency distribution of math scores across all 1,000 students, with 20 bins and a KDE (Kernel Density Estimate) curve overlaid in the same purple colour. Generated using **sns.histplot(df['math_score'], bins=20, kde=True, color='purple')**.

Observation: The distribution of math scores is **approximately bell-shaped (normal)**, centred around the 60–70 mark, which aligns with the computed mean of 66.09. The distribution has a slight left skew, with the left tail extending toward zero — reflecting the presence of a small number of very low scorers (including the student who scored 0). The KDE curve confirms the roughly unimodal, approximately symmetric shape of the distribution. Most students score between 50 and 80, with scores below 30 and above 95 being relatively rare. The peak of the distribution falls in the 65–70 bin, consistent with the median of 66.

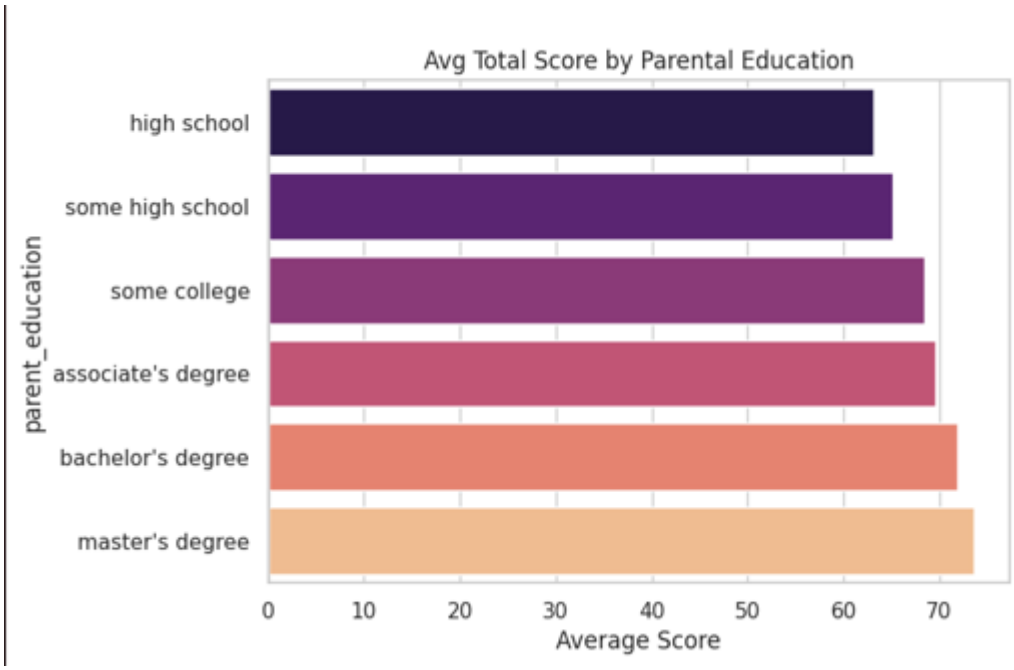
Chart 4: Average Scores by Gender



This grouped bar chart compares the average scores of female and male students across all three subjects — Math, Reading, and Writing. It was generated using **sns.barplot()** with the melted DataFrame (obtained via **scores_gender.melt()**), with gender as the hue variable and the palette set to 'Set2'.

Observation: The chart reveals a consistent and visually clear **cross-over pattern** in gender performance. Male students (salmon/orange bars) score **higher in Mathematics** (approximately 68.7 vs 63.6), while female students (teal/green bars) score **substantially higher in both Reading** (approximately 72.6 vs 65.5) and **Writing** (approximately 72.5 vs 63.3). The gender gap is largest in Writing, where females outperform males by approximately 9 points — the most pronounced subject-level performance difference in the entire dataset. This pattern is consistent with gender-based academic performance trends widely documented in educational research globally.

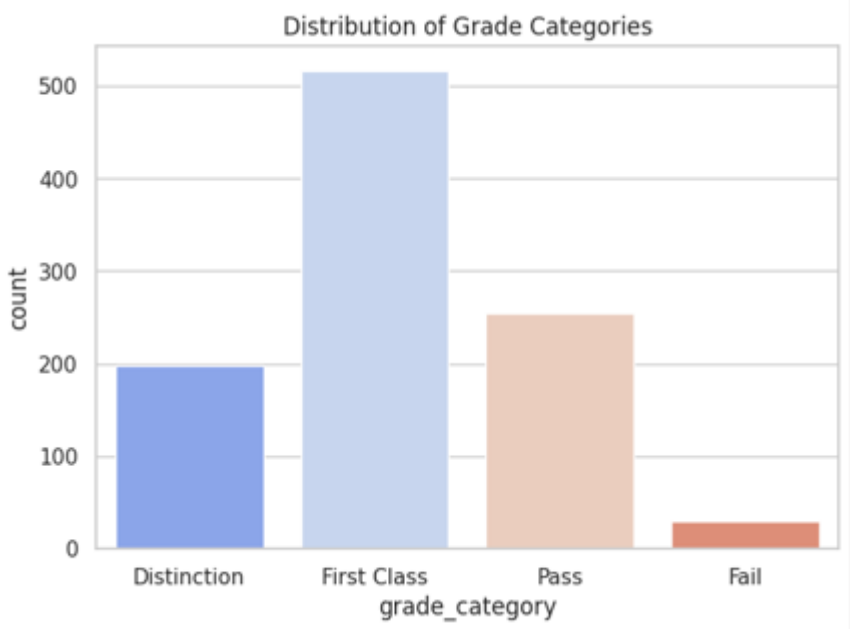
Chart 5: Avg Total Score by Parental Education



This horizontal bar chart displays the average total score (average_score) grouped by parental level of education, sorted in ascending order by average score. Generated using **sns.barplot()** with y=avg_by_edu.index and x=avg_by_edu.values, with the 'magma' palette applied.

Observation: A **clear upward progression** in average student scores is visible as parental education level increases. Students whose parents hold only a high school diploma have the lowest average scores, while students with master's degree parents achieve the highest averages. The gap between the lowest (high school) and highest (master's degree) parental education groups is approximately **10–11 score points** — a meaningful and substantial difference. This strongly supports the well-established hypothesis in educational research that parental education level is a significant predictor of a child's academic achievement. The progression from "some high school" to "bachelor's degree" is gradual and consistent, with each step up in parental education corresponding to a modest but meaningful improvement in average student scores.

Chart 6: Distribution of Grade Categories



This bar chart shows the count of students in each grade category — Distinction, First Class, Pass, and Fail — in order of achievement. Generated using **sns.countplot()** with the order parameter set to ['Distinction', 'First Class', 'Pass', 'Fail'] and the 'coolwarm' palette applied.

Observation: The distribution of grade categories is **clearly unimodal**, centred on the First Class category with **517 students (51.7%)**. This means that over half of all students fall in the 60–79 average score range. Distinction (198 students, 19.8%) is the second most common category, indicating a healthy proportion of high achievers. Pass (255 students, 25.5%) represents students in the 40–59 range who passed but may need academic support to reach First Class. Most encouragingly, only **30 students (3%) fall in the Fail category** — a very small minority of the overall population. The chart demonstrates that the dataset represents a generally well-performing student population with a realistic spread of achievement levels.

CONCLUSION

Through this EDA mini project on the Students Performance dataset, the following key conclusions can be drawn from the twelve EDA techniques applied:

Summary of Key Findings:

Finding	Result
Total Students Analysed	1,000
Average Math Score	66.09
Average Reading Score	69.17
Average Writing Score	68.05
Overall Average Score	67.77
Gender with Higher Math Score	Male (68.73 vs 63.63)
Gender with Higher Reading Score	Female (72.61 vs 65.47)
Gender with Higher Writing Score	Female (72.47 vs 63.31)
Largest Race/Ethnicity Group	Group C (319 students, 31.9%)
Smallest Race/Ethnicity Group	Group A (89 students, 8.9%)
Test Prep Completion Rate	35.8% (358 students)
Avg Score Gain from Test Prep (Writing)	~10 points
Students in Distinction	198 (19.8%)
Students in First Class	517 (51.7%)
Students in Pass	255 (25.5%)
Students Who Failed	30 (3.0%)
Standard Lunch Avg Score	~70.83
Free/Reduced Lunch Avg Score	~62.20
Missing Values	0 (dataset fully clean)

Finding	Result
Highest Parental Edu Group	Master's Degree (~73.7 avg)
Lowest Parental Edu Group	High School (~63.0 avg)

Key Conclusion 1 — Gender and Subject Performance: A clear and consistent cross-over pattern exists in gender performance. Male students outperform female students in Mathematics, while female students significantly outperform male students in both Reading and Writing — particularly in Writing, where the gap is nearly 9 points. This suggests that subject-specific gender support initiatives may be beneficial — for example, targeted math support for female students, and reading/writing encouragement for male students.

Key Conclusion 2 — Impact of Test Preparation: Completing the test preparation course is associated with approximately 6–10 point improvements across all three subjects. Given that only 35.8% of students completed the course, expanding access and participation in test preparation programs represents one of the highest-impact interventions available. The benefit is most pronounced in Writing, suggesting that structured writing practice (as offered in prep courses) yields the greatest return.

Key Conclusion 3 — Socioeconomic Proxy (Lunch Type): Students on the standard lunch plan outperform those on the free/reduced plan by approximately 8–11 points across subjects. This finding reinforces the well-established relationship between socioeconomic status and academic achievement. Policy interventions targeting resource support for economically disadvantaged students — including access to tutoring, materials, and test preparation — could help close this gap.

Key Conclusion 4 — Parental Education and Student Performance: There is a clear monotonic relationship between parental education level and student average scores. As parental education increases from high school to master's degree, average student scores increase consistently by approximately 10 points in total. This underscores the compounding generational effect of education — higher-educated parents tend to have children who achieve better academic outcomes.

Key Conclusion 5 — Grade Distribution: The majority of students (51.7%) fall in the First-Class category (60–79 average), with 19.8% achieving Distinction and only 3% failing. This indicates a generally healthy distribution of academic performance, with a large middle-performing group that could potentially be moved into the Distinction category through targeted academic support — particularly test preparation course access and socioeconomic support measures.

This project successfully demonstrated the application of all twelve EDA techniques taught in Semester 2 on a real-world student academic performance dataset. The

complete EDA pipeline — from data loading, inspection, and cleaning to analysis, grouping, filtering, and visualisation — was implemented in Python using Pandas, NumPy, Matplotlib, and Seaborn in a Jupyter Notebook environment. All twelve techniques produced meaningful, actionable insights about student performance patterns and the demographic and contextual factors that shape them.

End of Report

Submitted by: Ansh Yadav Faculty Incharge: Mrs. Snehal Bhosale Symbiosis Institute of Technology, Pune Department of Electronics & Telecommunication Engineering First Year B. Tech — Academic Year: 2025–26